

3.1 Overview of Association Rules

The Association rules are useful for analyzing and predicting customers behavior. They play an important role in applications like market basket analysis, customer-based analytics, catalog design, product clustering and store layout. It is a descriptive and unsupervised learning method to discover the relationships in a dataset and mining the transactions in databases.

The "Market-basket Analysis" model is commonly used for better understanding of association rules, where a many to many relationships between "items" and "baskets" are represented. In Market-basket Analysis, the large collection of transactions is given where each transaction comprises one or more associated items. In this analysis, the association rules go through each item being purchased to see what items are frequently bought together and to determine the list of rules that define the procuring behavior of a customer. It is one of the key techniques used by many online stores to uncover the associations between items which works by looking for combinations of items that frequently occurs together in a transaction. It identifies the strength of association between pairs of products purchased together and identify patterns of co-occurrence where co-occurrence is when two or more things take place together. It also allows retailers to recognize the relationships between the items or itemset that people buy. The term itemset is nothing but the collection of items or individual entities that contain some kind of relationship. The relationships are depends on both business context and the functionality of algorithm being used for the discovery.

Technically, the Market Basket Analysis creates If-Then scenario rules, for example, if item X is purchased then item Y is likely to be purchased. The rules are probabilistic in nature or, in other words, they are derived from the frequencies of co-occurrence in the observations. The rules can be used in pricing strategies, product placement, and various types of cross-selling strategies. The Market Basket Analysis takes data at transaction level, which lists all items bought by a customer in a single purchase. The technique determines relationships of what products were purchased with which other product(s). These relationships are then used to build profiles containing If-Then rules of the items purchased.

The rules for Market Basket Analysis could be written as :

IF {X} THEN {Y}

Where X and Y are the items or products

Here, If part of the rule (the {X} above) is known as the antecedent and the THEN part of the rule is known as the consequent (the {Y} above). The antecedent is the condition and the consequent are the result. For example, If customer is purchasing Bread then likely to purchase Butter also or likely to purchase Eggs.

Generally, association rule mining is the two-step process, first is finding all the frequent item sets and second is generate strong association rules from the frequent itemsets.

So, there are mainly two algorithms used for finding the frequent item sets namely Apriori algorithm and FP growth algorithm. In this chapter, we are mainly focusing on Apriori algorithms for the discussion of association rules.

3.2 Apriori Algorithm

The Apriori is one of the most fundamental algorithms for generating association rules. It uses support for pruning the itemsets and controlling the exponential growth of candidate itemsets where smaller candidate itemsets, which are known to be frequent itemsets, are combined and pruned to generate longer frequent itemsets.

This approach eliminates the need for all possible itemsets to be enumerated within the algorithm, since the number of all possible itemsets can become exponentially large. The major components of Apriori algorithm are Support and Confidence. For example, if 60 % of transactions contain itemset {Bread}, then the support of {Bread} is 0.6 and 50 % of all transactions contain itemset {Bread, Butter}, then the support of {Bread, Butter} would be 0.5.

A frequent itemset has items that satisfies minimum support criterion. If the minimum support is set at 0.5, any itemset can be considered a frequent itemset if at least 50 % of the transactions contain this itemset. In other words, the support of a frequent itemset must be greater than or equal to minimum support. Therefore, If the itemsets are considered as frequent, then any subset of the frequent itemset must also be frequent. This is referred to as the Apriori property. For example, if 50 % of the transactions contain {Bread, Jam}, then the support of {Bread, Jam} is 0.5, the support of {Bread} or {Jam} should be at least 0.5.

The Apriori algorithm takes a bottom-up iterative approach to discover the frequent itemsets by determining all the possible items and then identify which amongst them are frequent. Assuming the minimum support threshold is set to 0.5, the algorithm identifies and retains those itemsets that appear in at least 50 % of all transactions and discards or prunes the itemsets which are having support less than 0.5.

In first iteration, the Apriori algorithm, the identifies the frequent 1-itemsets (for example, {Bread}, {Milk}, {Butter}, ...) and evaluated to identify the frequent 1-itemsets among them. In the second iteration, the algorithm identifies frequent itemsets paired into 2-itemsets (for example, {Bread, Butter}, {Bread, Milk}, {Jam, Butter}, ...) and again evaluated to identify the frequent 2-itemsets among them. Likewise, algorithm continues to run n-iterations on n-itemsets to evaluate frequent n-itemsets among them. At each iteration, the algorithm checks whether the support criterion can be met and stops when algorithm runs out of support or until the itemsets reach a predefined length.

This algorithm uses two steps "join" and "prune" to reduce the search space. The join step generates (K+1) itemset from K-itemsets by joining each item with itself while prune step scans the count of each item in the database. If the candidate item does not meet minimum support, then it gets removed. This step is performed to reduce the size of the candidate itemsets.

The algorithm composed of sequence of steps to be followed to find the most frequent itemset in a given database which performs the join and the prune steps iteratively until the most frequent itemset is achieved. The generalized steps in Apriori algorithm are given as below

Apriori Algorithm
Step 1 : In the first iteration of the algorithm, each item is taken as a 1-itemsets candidate. The algorithm counts the occurrences of each item. Step 2 : In the set of 1-itemsets, the occurrence of items which satisfies the minimum support are determined. The minimum support is already defined in the problem. So, the candidates which count more than or equal to minimum support, are taken forward for the next iteration and the others are pruned or removed. Step 3 : In the second iteration, 2-itemset frequent items with minimum support are revealed. In this step the join of 2-itemset is generated by forming a group of 2 by combining items with itself. Step 4 : The 2-itemset candidates are pruned using minimum support threshold value as like step 2. Step 5 : The next iteration will form 3-itemsets using join and prune step. If all 2-itemset subsets are frequent then the superset will be frequent otherwise it is pruned. Step 6 : The process continues for {4-itemsets, 5-itemsets.... K-itemsets} till itemset does not meet the minimum support criteria or itemsets reach a predefined length.

Example 1:

In a given table the transaction Ids (TID's) and Itemsets are provided, so find all the frequent itemsets using Apriori algorithm with minimum support = 50 % and Minimum confidence = 50 %.

TID	Itemsets
T1	A, B, C
T2	A, C
T3	A, D
T4	B, E, F

As we have provided a minimum support = 50 %. Therefore, support count can be calculated as

$$\text{Support count} = \frac{50}{100} \times \text{No. of transactions} = \frac{50}{100} \times 4 = 2$$

Iteration 1 : Now, let us scan the dataset for each candidate (item) for getting its support count that means generating support count for each the candidates. Therefore, Candidate support calculated for 1-Itemset is shown in Table 3.2.1.

C1	
Items	Support_count
{A}	3
{B}	2
{C}	2
{D}	1
{E}	1
{F}	1

Table 3.2.1 : Candidate support for 1-Itemset

Now, compare candidate support count with minimum support count. Here, minimum support count = 2. So, from C1 compare support count of each item with minimum support count which should be greater than equal to 2. Therefore, we need to prune/remove the items whose support count is less than 2. So, the itemset that satisfies the minimum support count (L1) are shown in Table 3.2.2.

L1	
Items	Support_count
{A}	3
{B}	2
{C}	2

Table 3.2.2 : Items satisfying minimum support count for 1-Itemset in Iteration 1

Iteration 2 : Now perform Join operations by combining single items from L1 into 2-Itemsets with all combinations and find their support count. Therefore, candidate support (C2) calculated for 2-Itemset is shown in Table 3.2.3.

C2	
Items	Support_count
{A, B}	1
{B, C}	1
{A, C}	2

Table 3.2.3 : Candidate support for 2-Itemset

Compare candidate support count with minimum support count 2. Here, we find only one transaction {A, C} has support count equal to 2. Therefore, we need to prune/remove the other items whose support count is less than 2. The itemset that satisfies the minimum support count (L2) is shown in Table 3.2.4.

L2	
Items	Support_count
{A, C}	2

Table 3.2.4 : Items satisfying minimum support count for 2-Itemset in Iteration 2

Here, we find only one transaction which satisfies minimum support count and will be a frequent item set for given problem. Now, let us find out final rules with support and confidence for frequent itemsets are given in Table 3.2.5.

Association rule	Support count	Confidence	Confidence %
$A \rightarrow C$	2	= Support/Occurrence of A	66 %

		$= 2/3 = 0.66$	
$C \rightarrow A$	2	$= \text{Support/Occurrence of } C = 2/2 = 1$	100 %

Table 3.2.5 : Final rules with support and confidence

In, both the cases confidence is greater than minimum confidence 50 % given in the problem. Therefore, Final rules will be $A \rightarrow C$ and $C \rightarrow A$.

Example 2 :

In the given Table 3.2.6, the transaction Ids (TID's) and list of Itemsets are provided, so find all the frequent itemsets using Apriori algorithm with minimum support count is 2 and support confidence = 50 %.

TID	List of itemsets
T1	I1, I2, I5
T2	I2, I4
T3	I2, I3
T4	I1, I2, I4
T5	I1, I3
T6	I2, I3
T7	I1, I3
T8	I1, I2, I3, I5
T9	I1, I2, I3

Table 3.2.6 : Example data for Apriori algorithm

As per previous example, the different iterations of Apriori algorithm are given in following tables. Now let us find candidate support (C1) and Items satisfying minimum support count (L1) for 1-Itemset as shown in Table 3.2.7. In this example the minimum support count provided is 2, so prune all the values which are less than 2 in Support_count column.

C1	
Items	Support_count
{I1}	6
{I2}	7
{I3}	6
{I4}	2
{I5}	2

L1	
Items	Support_count
{I1}	6
{I2}	7
{I3}	6
{I4}	2
{I5}	2

Table 3.2.7 : Candidate support and items satisfying minimum support count for 1-itemset

Now calculate candidate support (C2) and Items satisfying minimum support count (L2) for 2-Itemset as shown in Table 3.2.8.

C2	
Items	Support_count
{I1, I2}	4
{I1, I3}	4
{I1, I4}	1
{I1, I5}	2
{I2, I3}	4
{I2, I4}	2
{I2, I5}	2
{I3, I4}	0
{I3, I5}	1
{I4, I5}	0

L2	
Items	Support_count
{I1, I2}	4
{I1, I3}	4
{I1, I5}	2
{I2, I3}	4
{I2, I4}	2
{I2, I5}	2

Table 3.2.8 : Candidate support and Items satisfying minimum support count for 2-Itemset

Similarly, calculate candidate support (C3) and Items satisfying minimum support count (L3) for 3-Itemset as shown in Table 3.2.9.

C2	
Items	Support_count
{I1, I2, I3}	2
{I1, I2, I5}	2

L2	
Items	Support_count
{I1, I2, I3}	2
{I1, I2, I5}	2

Table 3.2.9 : Candidate support and Items satisfying minimum support count for 3-Itemset

Now, let us find out final rules with support and confidence for frequent itemsets are given in Table 3.2.10.

Association rule	Support count	Confidence	Confidence %
I1 → I2 → I3	2	= Support/Occurrence of I1 = 2/6 = 0.33	33 %
I2 → I3 → I1	2	= Support/Occurrence of I2 = 2/7 = 0.28	28 %
I3 → I1 → I2	2	= Support/Occurrence of I3 = 2/6 = 0.33	33 %
I1 → I2 → I5	2	= Support/Occurrence of I1 = 2/6 = 0.33	33 %
I2 → I5 → I1	2	= Support/Occurrence of I2 = 2/7 = 0.28	28 %
I5 → I1 → I2	2	= Support/Occurrence of I5 = 2/2 = 1	100 %

Table 3.2.10 : Final rules with support and confidence

From above Table 3.2.10, we can conclude that the final frequent itemsets for given problem would be I5 → I1 → I2 as it has confidence more than minimum confidence 50 %.

3.3 Evaluation of Candidate Rules

The association rule has three important measures that express the degree of confidence in the rule called Support, Confidence, and Lift. The Support(S) is the number of transactions that include items in the {X} and {Y} parts of the rule as a percentage of the total number of transactions. It is a measure of how frequently the collection of items occur together as a percentage of all transactions.

The Confidence(C) is the ratio of the no of transactions that includes all items in {Y} as well as the no of transactions that includes all items in {X} to the no of transactions that includes all items in {X}.

Lift(L) of the rule $X \rightarrow Y$ is the confidence of the rule divided by the expected confidence, assuming that the itemsets X and Y are independent of each other. The expected confidence is the confidence divided by the frequency of {Y}. Leverage is a similar to Lift, but instead of using a ratio, leverage uses the difference. Here, Confidence

is able to identify trustworthy rules, but it cannot tell whether a rule is coincidental. Measures such as lift and leverage not only ensure interesting rules are identified but also filter out the coincidental rules.

The association rules discover all sets of items that have support more than the minimum support and then using the large itemsets to produce the desired rules that have confidence greater than the minimum confidence. The lift of a rule is the ratio of the observed support to that expected if X and Y are independent.

For Rule $X \rightarrow Y$, the Support, Confidence and Lift can be calculated as

$$\text{Support} = \frac{\text{Frequency}(X,Y)}{N} \text{ or } \frac{\text{Number of transactions in which (X,Y) appear}}{(\text{Total number of transactions})}$$

where N is total number of transactions

$$\text{Confidence} = \frac{\text{Frequency}(X,Y)}{\text{Frequency}(X)} \text{ or } \frac{\text{Support}(X,Y)}{\text{Support}(X)}$$

$$\text{Lift} = \frac{\text{Support}(X,Y)}{\text{Support}(X) * \text{Support}(Y)}$$

$$\text{Leverage} = \frac{\text{Support}(X,Y)}{\text{Support}(X) * \text{Support}(Y)}$$

For example, consider the following table which has list of Itemsets along with their transaction ID (TID). The support, confidence and lift can be calculated as follows

TID	ITEMS
T1	Bread, Milk
T2	Bread, Jam, Butter, Eggs
T3	Milk, Jam, Butter, Coke
T4	Bread, Milk, Jam, Butter
T5	Bread, Milk, Jam, Coke

From above table, If the Rule is $\{\text{Milk, Jam}\} \rightarrow \{\text{Butter}\}$

In above example, the combination of Milk, Jam and Butter has occurred 2 times while total number of transactions (N) = 5.

The support can be calculated as

$$\text{Support} = \frac{\text{Frequency}(X,Y)}{N}$$

Therefore, Support(S) of $(\{\text{Milk, Jam, Butter}\}) = \frac{2}{5} = 0.4$

Now Confidence can be calculated as

$$\text{Confidence} = \frac{\text{Frequency}(X,Y)}{\text{Frequency}(X)}$$

So, the confidence (C) for rule {Milk, Jam}→{Butter} would be

Confidence (C) = Frequency (Milk, Jam, Butter)/Frequency (Milk, Jam)

Therefore, Confidence (C) = 2/3 = 0.67

Now Lift can be calculated as

$$\text{Lift}(L) = \frac{\text{Support}(X,Y)}{\text{Support}(X)*\text{Support}(Y)}$$

L = Support ({Milk, Jam, Butter}) / Support ({Milk, Jam}) *Support({Butter})

Here, Support ({Milk, Jam, Butter}) = 0.4

Support ({Milk, Jam}) = 3/5=0.6 and Support({Butter}) =3/5 = 0.6

Therefore, Lift (L) = 0.4/ (0.6*0.6) = 1.11

Now, Leverage can be calculated as

Leverage = Support(X,Y)- (Support(X)*Support(Y))

= Support ({Milk, Jam, Butter}) – (Support ({Milk, Jam}) * Support({Butter}))

= 0.4 – (0.6*0.6)

= 0.04

3.4 Applications of Association Rules

There are many applications of association rules, some of them are described as below :

- 1. Market Basket Analysis :** This is the most common place example of association rules where data is gathered utilizing standardized barcode scanners in many supermarkets. This database, known as the " market basket " database, comprises of an enormous number of records on past transactions. A record in a database lists all the items purchased by a customer in one sale. Realizing which groups are inclined towards which set of items gives these shops the opportunity to alter the store layout and the store catalog to put the ideally concerning each other.
- 2. Medical Diagnosis :** Association rules in medical diagnosis can be beneficial for assisting physicians for curing patients. As, Diagnosis is not an easy process and has a scope of errors which may result in unreliable end-results. Using relational association rule mining, we can identify the probability of the occurrence of illness concerning various factors and symptoms. Further, it can be extended by adding

new symptoms and defining relationships between the new signs and the corresponding diseases.

3. **Census Data** : Every government has tonnes of census data which is used to plan efficient public services as well as help public businesses. This application of association rule mining helps governments for supporting complete public policy and bringing forth an efficient functioning of a democratic society.
4. **Protein Sequence** : The association rules can be used in proteins sequence findings. As proteins sequences made up of twenty types of amino acids. Each protein bears a unique 3D structure which depends on the sequence of these amino acids. A slight change in the sequence can cause a change in structure which might change the functioning of the protein. So, association rules can be effectively used to predict the exact protein sequence.
5. **Retail Marketplace** : In Retail, association rules can help determine what items are purchased together, purchased sequentially, and purchased by season. This can assist retailers to determine product placement and promotion optimization.
6. **Telecommunications** : In Telecommunications, association rules can be used to determine what services are being utilized and what packages customers are purchasing. For instance, Telecommunications these days is also offering TV and Internet. Creating bundles for purchases can be determined from an analysis of what customers purchase, thereby giving the company an idea of how to price the bundles.
7. **Banks** : In Financial organizations like Banks, the association rules can be used to analyze credit card purchases of customers to build profiles for fraud detection purposes and cross-selling opportunities.
8. **Insurance** : In Insurance, the association rule mining can be used to build profiles to detect medical insurance claim fraud by viewing profiles of claims. It allows to scan the profiles for determining whether a person has more than 1 claim belongs to a particular claimee within a specified period of time.

3.5 Finding Associations and Finding Similarity

There are various measures used to find the associations and similarity between the different entities. Finding association involves finding the frequent itemsets based on association rules using any of two methods like Apriori or FP-Growth methods. Here, association rules are used for finding those frequent itemsets whose support is greater or equal to a support threshold. So, if we are looking for association rules $X \rightarrow Y$ that apply

to a reasonable fraction of the baskets in Market-basket analysis, then the support of X must be reasonably high than Y. We also want the confidence of the rule to be reasonably higher to avoid any practical effects. So, for finding the association between the entities, we have to find all the itemsets that meet threshold of support and high confidence value. During the extraction of itemsets, we must be assumed that there are not too many frequent itemsets and thus not too many candidates for high-support, high-confidence association rules. This assumption leads to important consequences about the efficiency of algorithms. Therefore, it must be avoided to have too many candidates for high-support and high-confidence association rules.

3.5.1 Finding Similarity using Distance Measures

The similarity between entities can be measured by finding the closest distance between two data points. In Big data analytics, the similarity can be find using various distance measures. A distance measure is the measure of how similar one observation is compared to a set of other observations. The various distance measures used in big data analytics are explained as follows.

A) Euclidean distance :

The Euclidean distance is the distance between two points in Euclidean space. Considering two points, A and B, with their associated coordinates $\{a_1, a_2, \dots, a_n\}$ and $\{b_1, b_2, \dots, b_n\}$ the Euclidean distance between two points A and B is given by the formula

$$\text{distance } (A, B) = \sqrt{(a_1 - b_1)^2 + (a_2 - b_2)^2 + \dots + (a_n - b_n)^2}$$

The Euclidean distance between two points cannot be negative, because the positive square root is intended. Therefore, in Euclidean space we assume that lowers the distance between two points higher the similarity.

B) Jaccard similarity :

The Jaccard similarity is another measure for finding similarity index. The Jaccard similarity of sets A and B is nothing but the ratio of the sizes of the intersection and union of sets S and T. The Jaccard similarity is then given by the formula

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|}$$

Here, we consider that, lower the value of Jaccard distance higher the similarity.

C) Cosine similarity :

This method is very similar to the Jaccard similarity, but gives somewhat different results, because it measures similarity instead of dissimilarity. The cosine distance between two points is an angle that the vectors to those points make. This angle must be in the range of 0 degree to 180 degrees, regardless of how many dimensions that space has. The Cosine similarity can be calculated by the formula :

$$\cos(\theta) = \frac{A \cdot B}{||A|| ||B||} = \frac{\sum_{i=1}^n A_i B_i}{\sqrt{\sum_{i=1}^n A_i^2 \sum_{i=1}^n B_i^2}}$$

D) Edit distance :

The Edit distance is the distance between two strings $x = x_1, x_2 \dots x_n$ and $y = y_1, y_2 \dots y_m$ is the smallest number of insertions and deletions of single characters that will convert x to y . It is used to find how dissimilar two strings are by counting number of operations required to convert one string into another.

E) Hamming distance :

The Hamming distance is another measure for finding the similarity. In a given a space of vectors, we define the Hamming distance between two vectors to be the number of components in which they differ. The Hamming distance cannot be negative, and if it is zero, then the vectors are identical.

The above similarity measures are used in applications like plagiarism softwares where textual similarity is found between original document and plagiarized documents, mirror pages analysis to detect the fake webpages of popular websites where pages of these mirror sites are quite similar, but are rarely identical and so on.

3.6 Recommendation System

With the increase in use of internet, today most of the websites like Netflix, Amazon, YouTube etc. uses recommendation system to suggest relevant items for the searched contents to their users. It involves predicting user responses to options. Such a facility is called a recommendation system. Some of the examples of recommendation system are suggesting a news videos based on the searched topic by the users, offering articles to on-line newspaper readers based on a prediction of readers interest or offering customers a

suggestion about what they might like to buy, based on their past history of product searches or purchase in an on-line retailers website. In simple words, a recommendation system filters the data using different algorithms and recommends the most relevant items to users. It captures the past behavior of a customer and based on that, recommends the products which the users might be likely to buy.

The aim of developing recommender systems is to reduce information overload by retrieving the most relevant information and services from a huge amount of data, thereby providing personalized services. The most important feature of a recommender system is its ability to "guess" a user's preferences and interests by analyzing the behavior of this user and/or the behavior of other users to generate personalized recommendations. The purpose of a recommender system is to suggest relevant items to users. To achieve this task, there are four categories of methods exist namely collaborative filtering methods, content-based methods, knowledge based recommendation methods and hybrid recommendation methods.

There are four major types of recommendation systems shown in Fig. 3.6.1.

3.6.1 Classification of Recommendation Systems

There are four basic types of recommendation systems namely Collaborative, Content-based, Knowledge based and Hybrid as shown in Fig. 3.6.1.

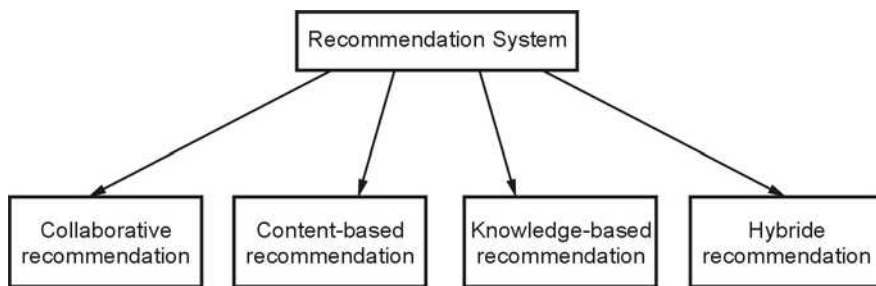


Fig. 3.6.1 : Classification of recommendation systems

The Collaborative recommendation systems aggregate ratings or recommendations of objects, recognize commonalities between the users on the basis of their ratings, and generate new recommendations based on inter-user comparisons. The Content-based recommendation system uses algorithms to recommends users about similar items that the user has liked in the past or is examining currently. The Knowledge based recommendation works on keeping functional knowledge about how a particular item meets a particular user need, and can therefore reason about the relationship between a need and a possible recommendation. This type of system makes suggestions based on

information relating to each user's preferences and needs. Using function knowledge, it can draw connections between a customer's need and a suitable product. While, the Hybrid recommender system combines the strengths of more than two recommender system and also eliminates any weakness which exist when only one recommender system is used. The detailed description about above recommendation systems are explained in next sections.

3.6.2 Applications of Recommendation Systems

Some of the popular applications of recommendation systems are given as follows

- a) **Recommendation of news articles** : The online news agencies uses recommendation system to identify articles of interest to readers, based on the articles that they have read in the past. The similarity might be based on the similarity of important words in the documents, or on the articles that are read by people with similar reading tastes.
- b) **Recommendations of movies** : These type of applications uses recommendations system to provide suggestion of movies user might like based on ratings provided by other users. The popular service providers who uses Movies recommendation are You tube, Netflix, Amazon Prime etc.
- c) **Recommendations of products** : It is one of the most popular application of recommendation system used by on-line retailers. It provides users with suggestions of products that they might like to buy based on the purchasing decisions made by similar customers. The popular online retailers are Amazon, eBay or Walmart etc. who uses recommendation system for their product recommendations.

3.7 Collaborative Recommendation

In collaborative recommendation, instead of using features of items to determine their similarity, similarity of the user ratings for two items has been found out. The process of identifying similar users and recommending what similar users like is called collaborative filtering. Collaborative recommendation techniques help to make choices based on the opinions of others who share similar interest. To make recommendations, it uses collaborative power of the ratings given by users. It uses two approaches for recommendations like user-based approach and item-based approach. In the user-based approach, a user receives recommendations of items liked by similar users while in the item-based approach, a user receives recommendations of items that are similar to those

they have liked in the past. The similarity between users or items can be calculated by correlation similarity or cosine-based similarity measures. When calculating the similarity between items using the above measures, only users who have rated both items are considered. The process of identifying similar users and recommending what similar users like is called collaborative filtering or collaborative recommendation system. It is represented as column in the utility matrix as shown in Fig. 3.7.1.

	Harry Potter 1	Harry Potter 2	Harry Potter 3	Jumanji
User 1	4		5	3
User 2	5	5	4	
User 3		4	3	4
User 4	4	4		4

Fig. 3.7.1 Utility matrix for collaborative recommendation

In above Fig. 3.7.1, The utility matrix represents movie ratings provided by different users for movies Harry Potter 1, Harry Potter 2, Harry Potter 3 and Jumanji on scale 1 to 5 where 1 is lowest and 5 is highest rating. There are many values which are blank in matrix, therefore collaborative recommendation system can be used to predict those missing values in the matrix.

The main challenge in designing collaborative recommender is that the ratings matrices are sparse in nature. Consider an example of a movie application in which users specify ratings with value like or dislike of specific movies. Most users would have watched only a small segment of the big number of available movies. As a result, most of the ratings are unspecified. The specified ratings are also referred to as observed ratings. The unspecified ratings will be called to as "missing" or "unobserved" ratings. The basic idea of collaborative filtering methods is that, these unspecified ratings can be estimated because the observed ratings are often highly correlated across various users and items. This similarity can be used to make the interpretations about partly specified values. Most of the models for collaborative filtering focus on leveraging either item-based correlations or user-based correlations for the prediction process. Few models use both types of correlations. This model is then used to estimate the missing values in the matrix, in the same way that a classifier imputes the missing test labels.

There are two methods that are effectively used in collaborative filtering namely Memory-based methods and Model-based methods. In Memory-based methods, the ratings of user and item combinations are predicted on the basis of their neighborhoods

because of that they are also called as neighborhood based collaborative filtering. Here, the similarity functions are computed between the columns of the ratings matrix to discover similar items. The upsides of memory-based techniques are that they are simple to implement and the resulting recommendations are easy to describe. On the other hand, memory-based algorithms do not work very well with sparse ratings matrices. Despite the fact that, the memory-based collaborative filtering algorithms are simple in nature but they tend to be heuristic and do not work well in all the circumstances.

In model-based methods, the combination of data mining and machine learning methods are utilized with regards to predictive models where the model is parameterized, the parameters of this model are learned within the context of an optimization framework. The examples of model-based methods include rule-based models, Bayesian methods, decision trees etc. In general, collaborative filtering is used for missing value analysis where underlying data matrix for the problem is very large and sparse.

3.8 Content-based Recommendation

The content-based recommendation system focuses on properties of items to find their similarity. It recommends the items based on a comparison between the content of the items and a user profile. In this technique, the descriptive attributes of items are used to make recommendations where the buying behavior of users and ratings are combined with the content information available in the items. The construction of each items profile is must where profile is a record or collection of records representing important characteristics of that item. The basic principles of Content-Based recommender systems are to analyze the description of the items preferred by a particular user, to determine the common attributes or preferences that can be used to distinguish these items stored in a user profile and to compare each item's preferences with the user profile so that only items that have a high degree of similarity matching with the users profile will be recommended.

Here, the item descriptions are often labeled with the ratings which are used as training data sets to create a user-specific regression or classification modeling. For each user, the training dataset stores the descriptions of the items that user has bought or rated while class variable is corresponding to the specified ratings or buying behavior. The content-based recommender systems use two techniques to generate recommendations. In first technique, the recommendations are generated heuristically using traditional information retrieval methods like cosine similarity measure while second technique, the recommendations are generated by building models using statistical learning and

machine learning approaches and are capable of learning users interests from the historical data of users.

The content-based methods are advantageous in making recommendations for new items which doesn't have sufficient number of ratings. For such items, the recommendations are made based on similarity index of other similar attributes which might have been rated by the users. Therefore, to make recommendations for such items, there is no requirement of history of ratings as supervised model will be able to leverage these ratings in combination with other items. The content-based methods have several disadvantages in many cases. First, the obvious recommendations are provided for the items which user has never consumed and has no chance of being recommended as it is undesirable because of the use of keywords. Second, even though content-based methods are effective in providing recommendations for new items, they are not reliable for new users. This is because the training model for the target user needs to have a large number of ratings available for the target item in order to make robust predictions without over fitting. Because of that the content-based methods have different trade-offs from collaborative filtering systems.

3.9 Knowledge based Recommendation

In Knowledge-Based (KB) recommendation system, the users are offered some items based on knowledge about the users, items and their relationships. It retains the functional knowledge base that defines how a particular item meets the specific user's requirement and, the exact recommendation is performed based on inferences about the relationship between a user's need and a possible recommendation. In this technique, the ontology is used as a knowledge representation method that represents the domain concepts and the relationships between those concepts. It is mostly used to express the domain knowledge and similarity between items based on the ontology. Most of the recommendation systems has limitation like cold start problem, which may arise due to the absence of information about ratings, user history, preferences, or clickstreams. It is difficult to resolve cold start problem due to the absence of information about user's preferences and lack of enough user ratings which makes it hard for predictions. As we know that, in collaborative and content-based systems, the recommendations are decided entirely by either the user's historic preferences, actions or ratings which becomes difficult when they are not available. In such cases, knowledge-based systems are used in which users are allowed to explicitly specify what they want.

The knowledge-based recommendations are mostly based on specific queries made by the user rather than user's rating history, they are mainly useful in the context of items that are not bought very often. Its examples include items belongs to financial services,

automobiles, real estate, tourism requests etc. In such examples, enough ratings may not be available for the recommendation process. As the items are bought rarely, and with different types of detailed options, it is difficult to obtain a sufficient number of ratings for a specific instantiation of the item at hand. This problem is also come across the cold-start problem. Furthermore, the historic data about such cases are not meaningful because of nature of consumer preferences may evolve over time when dealing with such items as particular item may have different attributes associated with its various properties, and a user may be attentive only in items with specific properties. For example, cars may have several makes, models, colors, engine options, and interior options, and user interests may be very specific based on combination of these options. Thus, in such cases the knowledge-based recommender systems can be useful, in which ratings are not used for the purpose of recommendations, rather, the recommendation process is performed on the basis of similarities between customer requirements and item descriptions, or the use of constraints specifying user requirements. The explicit specification of requirements results in greater control of users over the recommendation process. The knowledge-based recommender systems can be classified on the basis of the type of the interfaces used like constraint-based recommender systems and case-based recommender system. In constraint-based recommender systems, the users typically specify the requirements or constraints with lower or upper limits on the item attributes where domain-specific rules are used for matching the user's requirement for item attributes. While in case-based recommender systems the specific cases are specified by the user as targets or anchor points and similarity metrics are defined on the item attributes to retrieve analogous items to these cases. The similarity metrics form the domain knowledge that is used in such systems. The obtained results are often used as new target cases with some interactive modifications by the user. Note that in both the cases, the system offers an opportunity to the user to modify their specified requirements. However, the interactivity in knowledge-based recommender system is achieved using guidance takes place through either conversational systems, Search-based systems or navigation-based systems. In conversational systems, the preferences of users are determined iteratively in the context of a feedback loop as the item domain is complex and the user preferences can be determined only in the context of an iterative informal system. In search-based systems, the user preferences are formed by using a precise sequence of questions that can provide the ability to specify user constraints, while in navigation-based systems, the user specifies a number of modification requests to the item being currently recommended through an iterative set of change requests. Most of the times the knowledge-based systems work analogous to content-based systems but major difference between them is content-based systems learn from past user behavior, whereas

knowledge-based recommendation system recommends based on active user requirements and their interests.

3.10 Hybrid Recommendation Approaches

The hybrid recommendation technique combines the best features of two or more recommendation techniques into one to achieve higher performance and overcome the drawbacks of traditional recommendation techniques. Most of the recommendation systems now use a hybrid approach by combining collaborative filtering, content-based filtering, and other approaches. The Hybrid approaches can be applied in numerous ways: by making content-based and collaborative-based predictions separately and then combining them or by adding content-based abilities to a collaborative-based approach (and vice versa) or by merging the methods into a single model. These methods can also be used to overcome some of the common problems in recommender systems such as cold start and the sparsity problem, as well as the knowledge engineering bottleneck in knowledge-based approaches.

Netflix is the best example of hybrid recommender system. where website makes recommendations by associating the watching and searching ways of similar users (i.e. collaborative filtering) as well as by offering movies that share same characteristics with movies that a user has rated highly (i.e. content-based filtering).

Some of the hybrid recommendation techniques are :

- **Weighted** : Where recommendation system combines the score (Weight) of different recommendation components numerically and adjust the weighted predictions with outcomes confirmed or disconfirmed.
- **Switching** : Here, recommendation system performs switching between different recommendation system which is appropriate for current situation. Suppose the hybrid recommendation system has combination of content and collaborative based recommender systems. Initially, the switching hybrid recommender first deploy content-based recommender system and if it doesn't work then it deploys collaborative based recommender system.
- **Mixed** : In mixed mode, the recommendations from more than one technique are presented together, so that user can choose from a wide range of recommendations. Here, recommendations from other recommenders are used together to give the new recommendation.

- **Feature combination** : In this technique, the features are derived from different knowledge sources, then they are combined together and assigned to a single recommendation algorithm.
- **Feature augmentation** : This technique computes the feature or set of features, which are the part of the input for the next technique.
- **Cascade** : In this technique, the strict priority is given to all the recommenders where the lower priority recommenders flouting ties in the scoring of the higher ones.
- **Meta-level** : This technique uses only a single recommendation technique and produces some sort of model, which is used as a input for another technique.